Module 1 Guided Lab Session 3: Async and Resilience

Field	Value
Module	M1 C# 14 Essentials
Exercises	6 (Async/Await + Part B), 6B (Optional), 7 (Custom Exceptions), 7B (Integration Report)
Assessment Tiers	Tier 4 (LO 1.7, 1.8) + Tier 5 Integration (Exercise 7B)

Prerequisites Check Before You Start

Your project must have:

- Models.cs with EnrollmentRecord, Course, Student, IGradable, Quiz, LabAssignment (from Session 1)
- EnrollmentService.cs with ProcessRegistration using guard clauses and a switch expression (from Session 2)

If either file is missing or incomplete, go back and finish the previous session first every exercise below depends on those types.

Exercise 6: Connection Dropping Under Load (LO 1.7: Async/Await)

The situation: On the first day of registration, 200 students hit the enrollment endpoint simultaneously. The server froze. Not from CPU or memory pressure from thread starvation. The previous developer called .Result on every database query, blocking a thread pool thread for the entire duration of each 300ms database call. With 200 concurrent requests and only ~20 available threads, every thread was locked waiting for I/O. New requests queued. Timeouts cascaded. The registrar rebooted the server twice before calling engineering.

The fix is async/await. Instead of blocking a thread while waiting for the database, you release it back to the pool. When the database responds, the runtime picks up where you left off on any available thread. Same work, same result, but the thread pool never starves.

Step 1 See Thread Starvation in Numbers

In Program.cs, add:

```
using System.Diagnostics;
```

```
// Simulate 5 database calls, each taking 300ms

// THE WRONG WAY: Blocking with Thread.Sleep
var sw = Stopwatch.StartNew();
for (int i = 0; i < 5; i++)
{
    Thread.Sleep(300);  // Thread is HELD for 300ms  cannot serve anyone else
}
Console.WriteLine($"Blocking sequential: {sw.ElapsedMilliseconds}ms");

// ASYNC BUT STILL SEQUENTIAL: Thread released, but calls are one-at-a-time
sw.Restart();
for (int i = 0; i < 5; i++)
{
    await Task.Delay(300);  // Thread released while waiting  but still sequential
}
Console.WriteLine($"Async sequential:   {sw.ElapsedMilliseconds}ms");

// THE RIGHT WAY: Async parallel  all 5 start simultaneously
sw.Restart();
var tasks = Enumerable.Range(0, 5).Select(_ => Task.Delay(300));
await Task.WhenAll(tasks);
Console.WriteLine($"Async parallel:     {sw.ElapsedMilliseconds}ms");
```

Run it:

```
dotnet run
```

What you should see:

```
Blocking sequential: ~1500ms
Async sequential:    ~1500ms
Async parallel:      ~300ms
```

Three numbers. The first two are similar about 1500ms. The third is about 300ms. That is a 5x improvement. With 200 concurrent users, the difference between a server that responds and a server that crashes.

Step 2 Build the TMS Student Fetcher

Create a method that simulates loading a student from a database:

```
async Task<Student> FetchStudentAsync(string id)
{
    Console.WriteLine($"  Fetching {id}...");
    await Task.Delay(300);  // Simulate database latency
    return new Student
    {
        Id = id,
```

```
        Name = $"Student-{id}",
        Age = 20,
        GPA = id switch
        {
            "S1" => 3.8m,
            "S2" => 2.4m,
            "S3" => 3.5m,
            "S4" => 1.9m,
            "S5" => 3.2m,
            _ => 2.5m
        }
    };
}
```

Now add a second method that fetches a course:

```
async Task<Course> FetchCourseAsync(string code)
{
    Console.WriteLine($"  Fetching course {code}...");
    await Task.Delay(200);  // Simulate database latency
    return new Course
    {
        Code = code,
        Title = $"Course-{code}",
        Capacity = code switch
        {
            "CRS-101" => 2,
            "CRS-201" => 30,
            "CRS-301" => 15,
            _ => 25
        }
    };
}
```

Step 3 Load in Parallel

```
sw.Restart();

// Start all fetches simultaneously  students AND courses
string[] studentIds = ["S1", "S2", "S3", "S4", "S5"];
string[] courseCodes = ["CRS-101", "CRS-201", "CRS-301"];

var studentTasks = studentIds.Select(id => FetchStudentAsync(id));
var courseTasks = courseCodes.Select(code => FetchCourseAsync(code));

// Both arrays load concurrently
Student[] students = await Task.WhenAll(studentTasks);
Course[] courses = await Task.WhenAll(courseTasks);
```

```
Console.WriteLine($"\nLoaded {students.Length} students and {courses.Length} courses in {sw.ElapsedMilliseconds}ms");
foreach (var s in students)
{
    Console.WriteLine($"  {s.Name}  GPA: {s.GPA}");
}
```

What you should see: All 5 "Fetching…" messages appear almost simultaneously, and the total time is ~300ms (the time of the slowest single call), not ~1500ms.

Decision rule (use this in production code):

- Use `await` for every I/O operation (database, HTTP, file). Never call `.Result` or `.Wait()`.
- Use `Task.WhenAll` when you have multiple independent async operations that can run concurrently.
- Always return `Task` or `Task<T>` from async methods. Never use `async void` except for event handlers.

Trade-off you should know: `Task.WhenAll` is excellent for independent operations, but if one fails, you only see the first exception by default. For finer error handling per-task, iterate the task array after `WhenAll` and check each `.Exception` property.

Troubleshooting:

Problem	Likely cause	Fix
All times show ~1500ms	Using `await` inside a `for` loop instead of `Task.WhenAll`	Start all tasks first, then await them together
`AggregateException` from `Task.WhenAll`	One of the tasks threw	Wrap in `try/catch` and inspect `ex.InnerExceptions` for the individual failures
"Async method lacks 'await' operators" warning	You wrote `async` but never used `await` inside	Either add `await` to an operation or remove the `async` keyword
Elapsed time much higher than expected	Running in debug mode with breakpoints	Run without debugging, or use `dotnet run` from terminal

Exercise 6 Part B: The TMS Enrollment Engine

Now connect the pieces. You will load students in parallel, then attempt to enroll each one in a course tracking successes and failures.

```
var enrollCourse = new Course { Code = "CRS-101", Title = "C# Mastery", Capacity = 2 };
var enrollService = new EnrollmentService();

var enrollments = new List<EnrollmentRecord>();
var failures = new List<string>();

sw.Restart();

foreach (var student in students)
{
  try
  {
    var record = enrollService.ProcessRegistration(student, enrollCourse);
    enrollCourse.EnrolledCount++;
    enrollments.Add(record);
    Console.WriteLine($"  Enrolled: {student.Name}");
  }
  catch (InvalidOperationException ex)
  {
    failures.Add($"{student.Name}: {ex.Message}");
    Console.WriteLine($"  Rejected: {student.Name}  {ex.Message}");
  }
}
```

What you should see: The first 2 students enroll successfully. Students 3–5 are rejected because the course has reached capacity (Capacity = 2).

Troubleshooting:

Problem	Likely cause	Fix
All students enroll (none rejected)	Capacity is too high or EnrolledCount not incrementing	Set Capacity = 2 and increment EnrolledCount++ after each successful enrollment
ProcessRegistration throws ArgumentNullException	Student array contains nulls	Ensure FetchStudentAsync always returns a valid Student instance
Course capacity check not working	Guard clause missing in EnrollmentService	Check that ProcessRegistration throws when course.EnrolledCount >= course.Capacity

Exercise 6B: Safe Fire-and-Forget (Optional Not Assessed)

In the TMS, after a successful enrollment, the system should send a confirmation email. But sending an email takes time and should not block the enrollment response. The

temptation is to write _ = SendEmailAsync(student); discarding the Task. This is dangerous: if the email server is down, the exception is silently swallowed.

The safe pattern wraps the fire-and-forget logic in its own try/catch inside the method:

```
async Task SendConfirmationAsync(Student student)
{
    try
    {
        await Task.Delay(100);  // Simulate sending email
        Console.WriteLine($"  Email sent to {student.Name}");
    }
    catch (Exception ex)
    {
        // Log the failure  do NOT re-throw.
        // This is intentional fire-and-forget.
        Console.WriteLine($"  Email failed for {student.Name}: {ex.Message}");
    }
}
```

Read this pattern. You will use it in M4 and M7 for background notifications via SignalR.

Exercise 7: The Unhelpful Crash (LO 1.8: Exceptions & Custom Faults)

The situation: When the TMS loses connection to the regional database in Bahir Dar, the framework throws a generic SqlException with a 200-line stack trace. The registrar sees “An error occurred” on screen. The log file shows a raw exception dump that only a database administrator could parse. Nobody knows whether the problem is the network, the database server, or a malformed query.

Custom exceptions let your application speak the domain translating infrastructure failures into messages that operators and logs can act on. When the enrollment pipeline catches a SqlException, it wraps it in a TmsDatabaseException with context: which student, which course, which operation. The log now says something a human can fix.

Step 1 Create Two Custom Exceptions

Add to Models.cs (or a new Exceptions.cs file):

```
public class TmsDatabaseException : Exception
{
    public string Operation { get; }

    public TmsDatabaseException(string operation, string message)
        : base(message)
```

```
    {
        Operation = operation;
    }

    public TmsDatabaseException(string operation, string message, Exception innerException)
        : base(message, innerException)
    {
        Operation = operation;
    }
}

public class CapacityReachedException : InvalidOperationException
{
    public string CourseCode { get; }

    public CapacityReachedException(string courseCode)
        : base($"Course {courseCode} has reached maximum capacity.")
    {
        CourseCode = courseCode;
    }

    public CapacityReachedException(string courseCode, Exception innerException)
        : base($"Course {courseCode} has reached maximum capacity.", innerException)
    {
        CourseCode = courseCode;
    }
}
```

Step 2 Use Them in the Enrollment Pipeline

Update EnrollmentService.cs. Replace the capacity check InvalidOperationException with the new CapacityReachedException:

```
public class EnrollmentService
{
    public EnrollmentRecord ProcessRegistration(Student? student, Course? course)
    {
        if (student is null)
            throw new ArgumentNullException(nameof(student));
        if (course is null)
            throw new ArgumentNullException(nameof(course));
        if (course.EnrolledCount >= course.Capacity)
            throw new CapacityReachedException(course.Code);

        string standing = student.GPA switch
        {
            >= 3.5m => "Honors",
            >= 2.5m => "Good Standing",
```

```
            _ => "Academic Warning"
        };
        Console.WriteLine($"  {student.Name} is in {standing}.");

        return new EnrollmentRecord(student.Id, course.Code, DateTime.UtcNow);
    }
}
```

Notice that your `catch (InvalidOperationException ex)` block in Exercise 6 Part B still works because `CapacityReachedException` inherits from `InvalidOperationException`. The runtime matches the most specific applicable catch. If you want to access the `CourseCode` property, you can make the catch more specific:

```
// This already works (catches any InvalidOperationException, including CapacityReachedExcepti
on):
catch (InvalidOperationException ex)

// This is more precise (lets you access ex.CourseCode):
catch (CapacityReachedException ex)
```

Either version is correct. The precise version is better when you need the domain context.

Step 3 Catch Domain Exceptions

In `Program.cs`, test catching the custom exception:

```
try
{
    var overflowCourse = new Course { Code = "CRS-999", Title = "Overflow Test", Capacity = 0 };
    enrollService.ProcessRegistration(
        new Student { Id = "S99", Name = "Test", Age = 20, GPA = 3.0m },
        overflowCourse
    );
}
catch (CapacityReachedException ex)
{
    Console.WriteLine($"\nDomain exception caught:");
    Console.WriteLine($"  Course: {ex.CourseCode}");
    Console.WriteLine($"  Message: {ex.Message}");
}
```

Run it:

```
dotnet run
```

What you should see:

```
Domain exception caught:
  Course: CRS-999
  Message: Course CRS-999 has reached maximum capacity.
```

The calling code knows exactly which course is full and can display a meaningful message to the registrar not a generic "An error occurred."

Decision rule (use this in production code):

- Use built-in exceptions (ArgumentNullException, InvalidOperationException) for generic precondition failures.
- Create a custom exception only when domain context (course code, student ID, campus name) materially improves log readability or error handling.
- Always pass the original exception as innerException when wrapping infrastructure failures losing the root cause makes debugging impossible.

Troubleshooting:

Problem	Likely cause	Fix
Custom exception not caught by catch (CapacityReachedException)	Exception was thrown as base Exception type	Ensure you throw new CapacityReachedException(course.Code), not new Exception(...)
innerException is null	You used the single-parameter constructor	Use the two-parameter constructor when wrapping: new TmsDatabaseException(op, msg, ex)
"Type not found" for custom exceptions	File not saved or wrong namespace	Save the file containing the exceptions. In a single-project app, all files share the namespace

Exercise 7B: The Enrollment Report (LO 1.5 + 1.7 Integration)

The situation: Management wants a single console output that summarizes every enrollment run: how many students loaded, how many enrolled successfully, which ones failed and why, how long the whole thing took, and the class average GPA. This is the integration exercise that ties together everything from Exercises 1–7.

This exercise builds on the enrollment loop from Exercise 6 Part B. If you skipped Part B, go back and complete it first you need the enrollments and failures lists.

Implementation

Add this to the end of your Program.cs, after the enrollment loop from Exercise 6 Part B:

```
// Stop the timer
sw.Stop();
```

```
// Calculate class average GPA from loaded students
decimal classAverage = students.Length > 0
    ? students.Average(s => s.GPA)
    : 0m;

// Print the final report
Console.WriteLine("\n========== ENROLLMENT SUMMARY ==========");
Console.WriteLine($"Total students loaded:     {students.Length}");
Console.WriteLine($"Successful enrollments:    {enrollments.Count}");
Console.WriteLine($"Failed enrollments:        {failures.Count}");
Console.WriteLine($"Class average GPA:         {classAverage:F2}");
Console.WriteLine($"Total elapsed time:        {sw.ElapsedMilliseconds}ms");

if (failures.Count > 0)
{
    Console.WriteLine("\n--- Failure Details ---");
    foreach (var failure in failures)
    {
        Console.WriteLine($"  {failure}");
    }
}

Console.WriteLine("========================================");
```

What you should see:

```
========== ENROLLMENT SUMMARY ==========
Total students loaded:     5
Successful enrollments:    2
Failed enrollments:        3
Class average GPA:         2.96
Total elapsed time:        ~350ms

--- Failure Details ---
  Student-S3: Course CRS-101 has reached maximum capacity.
  Student-S4: Course CRS-101 has reached maximum capacity.
  Student-S5: Course CRS-101 has reached maximum capacity.
========================================
```

The elapsed time should be well under 2 seconds, proving that parallel loading worked. The failure messages now use `CapacityReachedException` from Exercise 7.

Troubleshooting:

Problem	Likely cause	Fix
stopwatch / sw is not defined	Variable declared in a different scope	Make sure Stopwatch.StartNew() is declared before the parallel load, and sw stays in scope through the report
enrollments or failures is not defined	Lists not declared before the enrollment loop	Declare var enrollments = new List<EnrollmentRecord>(); and var failures = new List<string>(); before the foreach
Elapsed time is very high (~1500ms+)	Students loaded sequentially, not in parallel	Use Task.WhenAll as shown in Exercise 6 Step 3
Class average GPA shows 0.00	Students array is empty	Verify FetchStudentAsync returns students with non-zero GPAs

Session 3 Checkpoint

Confirm all three:

- ☐ Loading 5 students + enrolling them completes in under 500ms total (parallel load working)
- ☐ Students 3–5 are rejected with CapacityReachedException messages that include the course code
- ☐ Enrollment summary report prints all five categories: loaded, successful, failed, average GPA, elapsed time

Lab Verification Full Module

Run your complete Program.cs one final time with dotnet run. Your output should demonstrate all nine core patterns:

1. **Null-Safe** string? with ?., ??, ??= operators (Exercise 1)
2. **Financially Precise** decimal arithmetic with no drift (Exercise 2)
3. **Immutable** EnrollmentRecord as a C# record (Exercise 3)
4. **Cleanly Validated** field keyword in Course.Capacity and Student.GPA (Exercise 3, Parts 2-3)
5. **Contract-Driven** IGradable interface with polymorphic PrintGradeReport (Exercise 3B)
6. **Readable Under Pressure** Guard clauses and switch expressions (Exercise 4)
7. **Declarative** LINQ with GroupBy, collection spread (Exercise 5)
8. **Asynchronous** Task.WhenAll for parallel loading (Exercise 6)

9. **Resilient** CapacityReachedException with domain context (Exercise 7)

If all nine are working, you have built a production-quality domain engine. Every type and pattern reappears in M4 (Web API), M5 (EF Core), and beyond.

Optional Extension: The Modular Audit Path (Delegates & Lambdas)

The situation: Every time a registration succeeds, the TMS must perform multiple side-effects: log the event to a file, send an SMS to the student, and update the campus leaderboard. If you hardcode these into the EnrollmentService, the class becomes a mess of dependencies that change for reasons unrelated to enrollment logic.

Using a delegate (or Action<T>) allows the service to announce that an enrollment happened without knowing what happens next. Other modules subscribe to the announcement and react independently.

Implementation (Complete Challenge)

No solution code is provided. Synthesize the logic from what you have learned:

```
// TODO 1: Define a delegate that accepts a Student object (or use Action<Student>)

public class EnrollmentService
{
  // TODO 2: Create a property that holds the delegate 'listener'

  public void FinalizeEnrollment(Student s)
  {
    Console.WriteLine("Persisting to database...");

    // TODO 3: Check if the delegate listener is 'not null'
    //       and invoke it with the student object.
  }
}

// TODO 4: In Program.cs, create a lambda function that prints:
//       "SMS SENT: Welcome to the TMS, [StudentName]!"
// TODO 5: Attach that lambda to the EnrollmentService and call FinalizeEnrollment.
```

What you should see:

Persisting to database...
SMS SENT: Welcome to the TMS, Abeba!

Troubleshooting:

Problem	Likely cause	Fix
NullReferenceException when invoking the delegate	No listener attached before calling FinalizeEnrollment	Assign your lambda to the service's delegate property first
Nothing prints after "Persisting to database…"	Null check is correct but lambda was not assigned	Assign the lambda: service.Listener = s => Console.WriteLine(...)

Lab Completion & Career Checkpoint

By completing Exercises 1–7B, you have resolved all 9 core C# 14 architectural anti-patterns assessed in M1. Your logic engine is now:

1. **Null-Safe** (Nullability enabled)
2. **Financially Precise** (Decimal Birr)
3. **Immutable** (Records)
4. **Cleanly Validated** (field keyword)
5. **Declarative** (LINQ + GroupBy + collection spread)
6. **Readable under pressure** (Guard clauses + switch expressions)
7. **Asynchronous** (Non-blocking Task management + parallel loading)
8. **Resilient** (Custom Exceptions TmsDatabaseException + CapacityReachedException)
9. **Integrated** (End-to-end enrollment report with Stopwatch timing)

Optional enhancement (not assessed in M1): 10. **Decoupled** (Delegates & Lambdas)

Appendix: Self-Paced Extension Activities

Total self-paced estimate: 90 minutes (6 activities x 15 minutes). Recommended use in full-time format: assign Activities 1-3 on Day 1 evening, 4-6 on Day 2 evening.

Activity 1 (15 min): Create a second console app called TmsCore.Tests using dotnet new console. Add a project reference from TmsCore.Tests to TmsCore using dotnet add reference. Verify both projects build with dotnet build.

Activity 2 (15 min): Build a currency converter that reads an amount in ETB, converts to USD and EUR using decimal rates, and prints the result formatted to 2 decimal places. Verify that switching to double produces a different (incorrect) result.

Activity 3 (15 min): Take 5 variables from your Session 1 code. Change each to string?. Fix every compiler warning using ?., ??, or ??=. Count how many potential null crashes you just prevented.

Activity 4 (15 min): Add a public override string ToString() method to every class you built in Session 1. Each should return a formatted summary (e.g., "Student: Abeba (STU-001), GPA: 3.8"). Verify that Console.WriteLine(student) now produces readable output.

Activity 5 (15 min): Using your student list from Session 2, write three new LINQ queries: (1) find all students whose name starts with “A”, (2) group students by their enrollment year, (3) calculate the total grant allocation for honors students only.

Activity 6 (15 min): Take any synchronous method from your Sessions 1-2 code. Convert it to return Task<T>, add await Task.Delay(100) to simulate latency, and call it from Program.cs using await. Verify it compiles and runs without warnings.